

Android Application Files — A Reference for FBO 3.0

What lives where, in what format, and how to enumerate it

2026-06-03

- [1. The mental model](#)
- [2. The distribution side: /data/app](#)
 - [2.1 The directory layout](#)
 - [2.2 The files, in detail](#)
 - [2.3 What's the total surface area?](#)
- [3. The runtime side: /data/data \(briefly\)](#)
- [4. The other relevant tree: /data/dalvik-cache](#)
- [5. Enumeration: how to get the file list](#)
 - [5.1 pm path — the shell primitive](#)
 - [5.2 dumpsys package — the verbose alternative](#)
 - [5.3 PackageManager API — when your code runs as an Android app](#)
 - [5.4 A complete shell-side enumeration script](#)
- [6. The complete FBO pin sequence, per file](#)
- [7. Lifecycle events that matter](#)
- [8. Verification on a development device](#)
- [9. Implications for FBO 3.0 architecture](#)
- [10. Summary](#)
- [11. Reference implementation](#)
 - [11.1 fbo_pin.c](#)
 - [11.2 fbo_enum.sh](#)
 - [11.3 Sample run on an x86 Linux box](#)
 - [11.4 End-to-end workflow on Android](#)
 - [11.5 What to change for production](#)

1. The mental model

An installed Android application exists in two completely separate places on disk, owned by different parts of the system and governed by different lifecycle rules. Conflating them is the single most common source of confusion when reasoning about file-level optimizations like FBO 3.0.

The distribution side — the files that came in with the APK install, the ART-compiled artifacts derived from them, and any extracted native libraries. These are read-only after installation, regenerated only on app update or system update, and dominate the I/O of cold app launch.

The runtime side — everything the app writes during normal operation: preferences, SQLite databases, downloaded content, caches, JIT compilation artifacts. These are read-write and their layout is dictated by the app itself.

For an app like Antutu (a benchmark), the cold-launch hot set is almost entirely on the distribution side. The benchmark scores reflect how quickly bytecode and native libraries can be loaded from storage — operations that exclusively touch `/data/app`. Pinning the distribution side is what FBO 3.0 wants. The data side is out of scope for this document.

2. The distribution side: /data/app

Every installed third-party application has its distribution files placed under `/data/app`, in a per-install directory whose path contains two randomized components inserted by `PackageInstaller`. The directory structure is otherwise completely regular.

2.1 The directory layout

```
/data/app/
├── <random-hash-1>/
│   ├── <package-name>-<random-hash-2>/
│   │   ├── base.apk
│   │   ├── split_config.<abi>.apk
│   │   ├── split_config.<density>.apk
│   │   ├── split_config.<locale>.apk
│   │   ├── lib/
│   │   │   ├── <abi>/
│   │   │   │   ├── libfoo.so
│   │   │   │   └── libbar.so
│   │   └── oat/
│   │       ├── <isa>/
│   │       │   ├── base.odex
│   │       │   ├── base.vdex
│   │       └── base.art
```

The two random hash components — let's call them `H1` and `H2` — are generated at install time. They change on every reinstall (including reinstalls triggered by app update). They exist to prevent other applications from constructing predictable paths into another app's install directory; this is part of the broader Android sandbox hardening that took shape from Android 8 onwards.

For your tooling, the critical implication is: **never hardcode paths under `/data/app`**. Always resolve them through `PackageManager`. The hashes are not predictable, but the structure beneath them is.

2.2 The files, in detail

base.apk

The primary APK. Despite the `.apk` extension, this is a standard ZIP archive. You can confirm this with `unzip -l base.apk` or `file base.apk`. Its contents follow a well-defined layout:

```
base.apk (ZIP)
├── AndroidManifest.xml
├── classes.dex
├── classes2.dex
├── classes3.dex
├── ...
├── resources.arsc
├── res/
│   ├── drawable-xxhdpi/
│   ├── layout/
│   └── ...
├── assets/
├── lib/
│   ├── <abi>/
│   │   └── *.so
├── META-INF/
│   ├── MANIFEST.MF
│   ├── CERT.SF
│   └── CERT.RSA
```

binary XML, not text
primary Dalvik bytecode
secondary DEX files
for apps that overflow the
single-DEX limit
compiled resource table
raw, uncompiled developer assets
only if extractNativeLibs=false
page-aligned, uncompressed
the signing certificate chain

What gets read at cold launch:

- `AndroidManifest.xml` is parsed once by `PackageManager` at install time, then again briefly at launch.
- `classes*.dex` are mmap'd by ART. On a fully AOT-compiled app, the `.dex` reads are minimal — most of the actual code execution comes from the `.odex` file in `oat/`. On a not-yet-compiled or interpretation-only app, the `.dex` reads dominate.
- `resources.arsc` is mmap'd and frequently accessed; resource lookups touch it constantly during UI inflation.
- Anything under `res/` is read on demand when the corresponding drawable / layout is first referenced.
- Anything under `lib/` is mmap'd if and when the app calls `System.loadLibrary`. For Antutu

specifically, this happens early.

`split_config.*.apk`

Android App Bundles (the modern distribution format since 2018) allow Play Store to deliver only the splits that match a given device. A device running on `arm64-v8a` with `xxhdpi` density and English locale might receive:

- `base.apk` — code and resources shared across all configurations
- `split_config.arm64_v8a.apk` — ABI-specific native code
- `split_config.xxhdpi.apk` — density-specific drawables
- `split_config.en.apk` — locale-specific strings

Each split is an APK in its own right (same ZIP structure), and ART loads them as a set. The Antutu install on your dev device may or may not have splits depending on how it was downloaded.

For FBO purposes, splits are pinned the same way as `base.apk`. The enumeration step from `pm path` returns the entire set in one call.

`oat/<isa>/base.odex`

This is where most of the cold-launch performance lives.

When an app is installed (or when `dex2oat` runs in the background due to “Optimizing apps” prompts after an OTA), Android compiles the DEX bytecode into native machine code for the device’s ISA. The result is an ELF binary placed at `oat/<isa>/base.odex`. ART maps this file directly into the app’s address space at launch and executes from it.

The `<isa>` directory uses the short instruction set name: `arm64` for 64-bit ARM, `arm` for 32-bit ARM, `x86_64`, `x86`. This is **not** the same as the ABI name used for native libs (where it’s `arm64-v8a`, `armeabi-v7a`, etc.) — a small inconsistency that catches everyone the first time.

The compilation level varies. Possible states for any given `.odex`:

- `speed` — fully AOT compiled. Fastest startup, biggest file.
- `speed-profile` — profile-guided AOT, only hot methods compiled. Smaller, still fast for the methods that matter.
- `quicken` — minimal compilation, just bytecode quickening.
- `verify` — verified only, no compilation.

Modern Android leans heavily on `speed-profile`: the system collects a profile of which methods the app actually executes, then re-compiles during idle/charging hours with that profile as input. This means the `.odex` file may be **rewritten** weeks after the app was installed, without any user-visible action. This is a wrinkle you will need to handle if you keep pins valid long-term — file mtime or inode change on the `.odex` is your signal that the previous pin record is stale.

`oat/<isa>/base.vdex`

Verified DEX. Contains the original DEX bytecode plus the verifier’s metadata. ART uses this as a fallback (for methods not present in `.odex`) and for verification. Smaller than the raw `.dex` files would be, mmap’d at launch.

`oat/<isa>/base.art`

The boot image / class preloader. A pre-cooked memory image containing pre-resolved class objects, string-interning tables, and other ART runtime structures. Loaded very early in the app launch sequence — before any application code runs — and provides much of the warm-start speedup over a cold ART boot.

If you had to pick a single file whose pinning gives the most cold-start benefit per pinned byte, `.art` is the candidate. It’s small, it’s the first thing read, and its access pattern is read-once-sequential which is the worst case for unpinned flash on a phone that just woke up.

`lib/<abi>/*.so`

Native libraries, extracted to disk at install time, **if** the app manifest has

```
android:extractNativeLibs="true" .
```

Since Android 6, the default has been to *not* extract — instead the `.so` files live inside `base.apk` (or inside a `split_config` APK), page-aligned and stored uncompressed, and ART mmap's them directly out of the APK at the right offsets. This is more efficient (no duplicated data on disk) and is the modern default.

For the FBO 3.0 enumeration, you handle both cases:

- If `lib/<abi>/` exists under the install dir: pin those `.so` files as separate entries.
- If it does not exist: the libs are inside `base.apk` (or splits), so pinning the APK covers them.

`dumpsys package <package>` reports `legacyNativeLibraryDir` and `primaryCpuAbi` from which you can derive the exact native lib directory if extraction happened.

2.3 What's the total surface area?

For a typical mid-sized app on a modern device:

Component	Typical size
<code>base.apk</code>	20 – 200 MB
Splits (total)	5 – 50 MB
<code>base.odex</code>	similar size to <code>classes*.dex</code> in the APK, often 10 – 100 MB
<code>base.vdex</code>	5 – 50 MB
<code>base.art</code>	1 – 10 MB
Extracted libs (if any)	5 – 100 MB

Antutu specifically, given its size and the embedded benchmark payloads, sits at the higher end of these ranges. A reasonable upper bound for the full distribution-side footprint of a single benchmark-sized app: 300–500 MB.

For an OEM building an FBO 3.0 feature, the UFS pin region budget needs to be sized with this in mind. A pin region of, say, 1 GB can fit two to three apps' worth of distribution files comfortably; an LRU or LFU eviction policy decides which apps remain pinned as the user runs different things.

3. The runtime side: `/data/data` (briefly)

The runtime data tree, mentioned for completeness and to make clear why you exclude it from FBO scope:

```
/data/data/<package-name>/
├─ files/           # arbitrary files the app writes
├─ cache/           # caches; the system may clear on pressure
├─ databases/       # SQLite DBs
├─ shared_prefs/    # key-value XML preferences
├─ code_cache/      # JIT artifacts, JIT-PGO profiles
├─ no_backup/       # opt-out of cloud backup
└─ app_<name>/     # arbitrary sub-directories the app creates
```

For benchmark and game-perf use cases, the data tree is overwhelmingly write-heavy or cold-read. Pinning it would consume budget without yielding the perf signal that benchmarks measure. Hence the scope decision in FBO 3.0 to ignore it.

For other product use cases (browser cache, chat app message database) the data tree matters a lot — but that's a different feature with different lifecycle rules and is explicitly out of scope here.

4. The other relevant tree: /data/dalvik-cache

On some Android versions and for some apps (particularly system apps under `/system/app` and `/system/priv-app`), the ART artifacts live not next to the APK in `oat/`, but in a central `/data/dalvik-cache` tree:

```
/data/dalvik-cache/
├── <isa>/
│   ├── system@app@<AppName>@<AppName>.apk@classes.dex
│   ├── system@app@<AppName>@<AppName>.apk@classes.odex
│   ├── system@app@<AppName>@<AppName>.apk@classes.vdex
│   └── system@app@<AppName>@<AppName>.apk@classes.art
```

The filename encoding takes the original APK path and replaces `/` with `@`. This is legacy from the original Dalvik VM. Modern third-party apps (Play Store installs) keep their OAT artifacts under `/data/app/.../oat/`, but you may still see `/data/dalvik-cache` entries for boot-class-path artifacts (the framework's own OAT files that live under the system server).

For FBO scope, you generally do not pin from `/data/dalvik-cache` unless you're targeting system apps. For the Antutu case, all the relevant OAT is under `/data/app/.../oat/`.

5. Enumeration: how to get the file list

There are several layers of API for resolving "what are the files for this app?". From most-to-least-shell-friendly:

5.1 `pm path` — the shell primitive

```
pm path com.antutu.ABenchMark
```

Output:

```
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/base.apk
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/split_config.arm64_v8a.apk
package:/data/app/<H1>/com.antutu.ABenchMark-<H2>/split_config.xxhdpi.apk
```

One line per APK (base + each installed split), prefixed with `package:` . Strip the prefix and you have absolute paths.

This call goes through `PackageManagerService` and uses the same internal data structures Android itself uses to find the files at launch. You will not get a wrong answer from it.

5.2 `dumpsys package` — the verbose alternative

```
dumpsys package com.antutu.ABenchMark
```

A much larger output, but contains everything: install location, data dir, native lib dir, primary ABI, version codes, granted permissions. The fields of interest for FBO are:

```
codePath=/data/app/<H1>/com.antutu.ABenchMark-<H2>
resourcePath=/data/app/<H1>/com.antutu.ABenchMark-<H2>
legacyNativeLibraryDir=/data/app/<H1>/com.antutu.ABenchMark-<H2>/lib
primaryCpuAbi=arm64-v8a
```

`codePath` is the install directory. Append `/oat/<isa>` to get to the OAT artifacts.

`legacyNativeLibraryDir` is set whether or not extraction occurred; check whether the directory actually

exists and is non-empty.

5.3 PackageManager API — when your code runs as an Android app

```
val pm = context.packageManager
val ai: ApplicationInfo = pm.getApplicationInfo("com.antutu.ABenchMark", 0)

val files = mutableListOf<String>()
files += ai.sourceDir // base.apk
ai.splitSourceDirs?.let { files += it } // splits
files += File(ai.nativeLibraryDir).listFiles()
    ?.map { it.absolutePath } ?: emptyList()

val installDir = File(ai.sourceDir).parent
val abi = Build.SUPPORTED_ABIS[0] // e.g. "arm64-v8a"
val isa = when (abi) {
    "arm64-v8a" -> "arm64"
    "armeabi-v7a" -> "arm"
    "x86_64" -> "x86_64"
    "x86" -> "x86"
    else -> abi
}
files += File("$installDir/oat/$isa").listFiles()
    ?.map { it.absolutePath } ?: emptyList()
```

This is the form your eventual controller daemon will use if it ships as a privileged Android service rather than a CLI tool.

5.4 A complete shell-side enumeration script

For your prototype, this is sufficient:

```
#!/system/bin/sh
PKG=$1
[ -z "$PKG" ] && { echo "usage: $0 <package>"; exit 1; }

ABI=$(getprop ro.product.cpu.abi)
case "$ABI" in
    arm64-v8a) ISA=arm64 ;;
    armeabi-v7a) ISA=arm ;;
    x86_64) ISA=x86_64 ;;
    x86) ISA=x86 ;;
    *) ISA=$ABI ;;
esac

# APKs (base + splits)
APKS=$(pm path "$PKG" 2>/dev/null | sed 's/^package:/' )
[ -z "$APKS" ] && { echo "package not installed: $PKG"; exit 1; }

# Install directory derived from first APK path
DIR=$(dirname "$(echo "$APKS" | head -1)")

echo "$APKS"
ls "$DIR/oat/$ISA"/* 2>/dev/null # base.odex, base.vdex, base.art
ls "$DIR/lib/$ABI"/* 2>/dev/null # only if extracted
```

This is the complete static pinnable surface for one package. For Antutu, expect a list of 8 to 20 files.

6. The complete FBO pin sequence, per file

Given a single file path resolved by the enumeration above, the sequence to make it pinned and durable is:

```

int fd = open(path, O_RDONLY);

// Step 1: prevent F2FS GC from relocating this file's blocks.
// Without this, the FIEMAP we take next can become stale at any
// time the cleaner runs.
int pin = 1;
ioctl(fd, F2FS_IOC_SET_PIN_FILE, &pin);

// Step 2: get the (logical_offset -> physical_LBA, length) map.
struct fiemap fm = {
    .fm_start      = 0,
    .fm_length     = ~0ULL,
    .fm_flags      = FIEMAP_FLAG_SYNC,
    .fm_extent_count = N,
};
// ... allocate room for N fiemap_extent entries after fm ...
ioctl(fd, FS_IOC_FIEMAP, &fm);

// Step 3: hand the physical LBA ranges to the UFS pin command.
// Interface varies by vendor – sysfs node, scsi passthrough, etc.
for (each extent in fm.fiemap_extents) {
    ufs_vendor_pin(extent.fe_physical, extent.fe_length);
}

// Step 4: record what was pinned, so unpin can reverse it later.
record_pin(package, path, inode, mtime, extents);

close(fd);

```

The unpin path reverses the same sequence: walk recorded extents, issue the vendor unpin command, optionally clear the F2FS pin if no other consumer needs it.

7. Lifecycle events that matter

A pinned-files registry has to be aware of the events that invalidate its records. These are all observable from a service listening to `PackageManager` broadcasts and from filesystem-level metadata checks.

Event	Mechanism to observe	Action
App installed	<code>ACTION_PACKAGE_ADDED</code>	Optional: pin if policy says so
App updated	<code>ACTION_PACKAGE_REPLACED</code>	Unpin old extents, re-enumerate, re-pin
App removed	<code>ACTION_PACKAGE_REMOVED</code>	Unpin extents
OAT regenerated by <code>dex2oat</code>	inode/mtime change on <code>.odex</code>	Re-FIEMAP and re-pin OAT subset
System OTA	reboot + <code>dex2oat</code> runs for all apps	Re-pin everything tracked
Manual <code>pm clear-package</code>	<code>ACTION_PACKAGE_DATA_CLEARED</code>	No action (only affects data dir)
F2FS GC moves blocks	should not happen if <code>F2FS_IOC_SET_PIN_FILE</code> is set	If somehow it does: detect via stale-extent reads, re-FIEMAP

The OAT-regeneration case is the easy one to forget. Android runs `dex2oat` opportunistically based on collected profiles, on charge, on idle, after OTAs, and at install. Your records need an integrity check before being trusted — comparing recorded inode and mtime against the current file’s metadata is sufficient.

8. Verification on a development device

Before writing any pinning code, run these on the dev target to confirm everything lines up with this document. Substitute any installed third-party app for `<pkg>`.

```
# Pick a target
PKG=com.antutu.ABenchMark
pm list packages | grep antutu      # confirm install name

# Resolve the file set
pm path $PKG
DIR=$(dirname $(pm path $PKG | head -1 | cut -d: -f2))
echo $DIR
ls $DIR
ls $DIR/oat/*
ls $DIR/lib/* 2>/dev/null || echo "(no extracted libs – they're inside the APK)"

# Pick one file and confirm we can FIEMAP it
F=$DIR/base.apk
filefrag -e $F | head
# Or, more specifically, use the FIEMAP ioctl from a small C program

# Confirm F2FS pin is available on this kernel
ls /sys/fs/f2fs/                  # should list one entry per F2FS mount
grep -i pin /proc/$(pidof <something on f2fs>)/mountinfo
# Try the ioctl from a small program; success indicates the kernel supports it

# Strace an app launch to see what it actually opens
am force-stop $PKG
strace -f -e openat -o /sdcard/antutu_strace.log am start -n $PKG/<MainActivity>
# Then analyze: which paths matter, what fraction are under /data/app vs /data/data
```

The strace output is the empirical confirmation that the distribution-side enumeration covers what actually matters. For a benchmark like Antutu, expect the overwhelming majority of `openat` calls during launch to be on files under the install directory and on framework files under `/system` (which you typically don't pin per-app).

9. Implications for FBO 3.0 architecture

Restating the design choices that fall out of the above, for the record:

The pinnable file set is statically known per installed app. It is enumerable in milliseconds via `pm path` + a directory listing. No runtime observation of `open()` calls is required to discover it.

The set is stable between app launches. Inodes and physical extents do not change as long as F2FS GC is prevented from moving the files (which `F2FS_IOC_SET_PIN_FILE` ensures). Therefore the pin can be applied at any time before the app is launched — at install, on user request, on predictive trigger — and remains effective.

Invalidation is event-driven and small in scope. Three triggers (package replaced, package removed, OAT regenerated) cover every real-world reason a pin record becomes stale. All three are observable without polling.

No `open()` interception is required for the static-file pin use case. The fanotify / kernel-hook design is correct only for use cases where the relevant file set is dynamic (e.g., apps that download gigabytes of content at first launch, where the downloaded files are themselves the perf-critical reads). For benchmark and standard-app acceleration, the static enumeration is complete.

Pin region budget is the real product constraint, not file discovery. UFS pin region size is finite (typically tens of MB to a few GB depending on device). The interesting engineering work is the eviction policy: which app's files stay pinned when the user installs the eighth game in a row. This is a cache-management problem, not an I/O-interception problem.

10. Summary

The Android application file layout, for the purposes of FBO 3.0, is:

- **Distribution files** live under `/data/app/<H1>/<package>-<H2>/` with predictable structure but randomized hashes.
- The hot set is `base.apk`, `split_config.*.apk`, `oat/<isa>/*`, and optionally `lib/<abi>/*.so` if extracted.
- The set is enumerable via `pm path` and one directory listing.
- The set is stable between launches; invalidated only by app update, uninstall, or OAT regeneration.
- F2FS pinning before FIEMAP is required to keep the pin durable against background GC.
- No syscall interception is needed for this use case; pin is a proactive, persistent operation on the storage device.

The next concrete step, on the rooted dev target, is to run the enumeration script in section 5.4 against an installed app, then a small C program that performs the FIEMAP step (without the UFS pin command for now) and prints the resulting extent list. Once the extent list looks correct, the only remaining variable is the vendor UFS pin interface, which is OEM-specific and outside the scope of this document.

11. Reference implementation

This section contains a complete working reference for the static-pin flow described in this document. It is written so it compiles and runs on an x86 Linux box (where the UFS pin step is stubbed and F2FS pin is silently no-op'd on ext4) and ports to Android with no source changes once the `ufs_vendor_pin()` function is wired to the real OEM pin interface.

Two files:

- **fbo_pin.c** — the per-file pin/unpin/list tool. Performs the F2FS pin, FIEMAP, UFS pin (stub), and record-keeping for one or more files passed on the command line.
- **fbo_enum.sh** — Android-side shell wrapper that resolves a package name to its distribution file set using `pm path` and a directory listing, and prints one path per line. Pipe its output into `fbo_pin pin`.

11.1 fbo_pin.c

```
/*
 * fbo_pin.c - FBO 3.0 static-pin reference implementation.
 *
 * Build: gcc -O2 -Wall -o fbo_pin fbo_pin.c
 *
 * Usage:
 * sudo ./fbo_pin pin <record-file> <file> [<file>...]
 * sudo ./fbo_pin unpin <record-file>
 * ./fbo_pin list <record-file>
 */

#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <errno.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/ioctl.h>
#include <sys/stat.h>
#include <linux/types.h>
#include <linux/fs.h>
#include <linux/fiemap.h>

/* F2FS pin ioctl. The number is part of the kernel ABI and stable.
 * Defined locally because <linux/f2fs.h> isn't always packaged. */
```

```

#ifdef F2FS_IOC_SET_PIN_FILE
#define F2FS_IOC_SET_PIN_FILE _IOW(0xf5, 13, __u32)
#endif

#define MAX_EXTENTS 1024

/* Step 1: tell F2FS not to relocate this file's blocks during GC.
 * Silently ignored on non-F2FS filesystems (ENOTTY). */
static int do_f2fs_pin(int fd, const char *path, int pin)
{
    __u32 v = pin ? 1 : 0;
    if (ioctl(fd, F2FS_IOC_SET_PIN_FILE, &v) < 0) {
        if (errno != ENOTTY)
            fprintf(stderr, " warning: F2FS pin on %s: %s\n",
                    path, strerror(errno));
        return -1;
    }
    return 0;
}

/* Step 2: read the (logical -> physical LBA, length) extent map. */
static int fiemap_file(int fd, const char *path,
                      struct fiemap_extent *out, unsigned *out_n)
{
    size_t bytes = sizeof(struct fiemap) +
                  MAX_EXTENTS * sizeof(struct fiemap_extent);
    struct fiemap *fm = calloc(1, bytes);
    if (!fm) { perror("calloc"); return -1; }

    fm->fm_start = 0;
    fm->fm_length = ~0ULL;
    fm->fm_flags = FIEMAP_FLAG_SYNC;
    fm->fm_extent_count = MAX_EXTENTS;

    if (ioctl(fd, FS_IOC_FIEMAP, fm) < 0) {
        fprintf(stderr, " FIEMAP on %s: %s\n",
                path, strerror(errno));
        free(fm);
        return -1;
    }
    unsigned n = fm->fm_mapped_extents;
    if (n > MAX_EXTENTS) n = MAX_EXTENTS;
    memcpy(out, fm->fm_extents, n * sizeof(struct fiemap_extent));
    *out_n = n;
    free(fm);
    return 0;
}

/* Step 3: ship LBA ranges to the UFS pin command. Stubbed here -
 * on a real device, replace this with the OEM's pin interface
 * (sysfs write, scsi passthrough, or vendor ioctl). */
static int ufs_vendor_pin(const char *path,
                        const struct fiemap_extent *exts, unsigned n,
                        int pin)
{
    const char *op = pin ? "PIN" : "UNPIN";
    (void)path;
    for (unsigned i = 0; i < n; i++) {
        printf(" ufs_%s lba=0x%llx len=0x%llx flags=0x%x\n",
               op,
               (unsigned long long)exts[i].fe_physical,
               (unsigned long long)exts[i].fe_length,
               exts[i].fe_flags);
    }
    return 0;
}

/* Step 4: persist what we pinned so unpin can reverse it. */
static int record_write(FILE *rec, const char *path,
                      ino_t ino, time_t mtime,
                      const struct fiemap_extent *exts, unsigned n)

```

```

{
    fprintf(rec, "FILE %s ino=%llu mtime=%lld nextents=%u\n",
            path, (unsigned long long)ino,
            (long long)mtime, n);
    for (unsigned i = 0; i < n; i++) {
        fprintf(rec, "  EXT phys=0x%llx len=0x%llx flags=0x%x\n",
                (unsigned long long)exts[i].fe_physical,
                (unsigned long long)exts[i].fe_length,
                exts[i].fe_flags);
    }
    return 0;
}

static int cmd_pin(const char *recpath, char **paths, int npaths)
{
    FILE *rec = fopen(recpath, "w");
    if (!rec) { perror(recpath); return 1; }

    int ok = 0;
    for (int i = 0; i < npaths; i++) {
        const char *path = paths[i];
        printf("PIN %s\n", path);

        int fd = open(path, O_RDONLY);
        if (fd < 0) {
            fprintf(stderr, "  open: %s\n", strerror(errno));
            continue;
        }
        struct stat st;
        if (fstat(fd, &st) < 0) { perror("  fstat"); close(fd); continue; }

        do_f2fs_pin(fd, path, 1);

        struct fiemap_extent exts[MAX_EXTENTS];
        unsigned n = 0;
        if (fiemap_file(fd, path, exts, &n) < 0) { close(fd); continue; }

        ufs_vendor_pin(path, exts, n, 1);
        record_write(rec, path, st.st_ino, st.st_mtime, exts, n);

        close(fd);
        printf("  done - %u extent(s) pinned\n\n", n);
        ok++;
    }
    fclose(rec);
    printf("pinned %d / %d files; record at %s\n", ok, npaths, recpath);
    return 0;
}

static int cmd_unpin(const char *recpath)
{
    FILE *rec = fopen(recpath, "r");
    if (!rec) { perror(recpath); return 1; }

    char line[1024], path[512] = {0};

    /* First pass: unpin every recorded extent. */
    while (fgets(line, sizeof(line), rec)) {
        if (strncmp(line, "FILE ", 5) == 0) {
            sscanf(line, "FILE %511s", path);
            printf("UNPIN %s\n", path);
        } else if (strncmp(line, "  EXT ", 6) == 0) {
            struct fiemap_extent e = {0};
            unsigned long long phys, len; unsigned flags;
            if (sscanf(line, "  EXT phys=0x%llx len=0x%llx flags=0x%x",
                    &phys, &len, &flags) == 3) {
                e.fe_physical = phys;
                e.fe_length = len;
                e.fe_flags = flags;
                ufs_vendor_pin(path, &e, 1, 0);
            }
        }
    }
}

```

```

    }
}

/* Second pass: clear the F2FS pin so the file is GC-eligible again. */
rewind(rec);
while (fgets(line, sizeof(line), rec)) {
    if (strncmp(line, "FILE ", 5) != 0) continue;
    sscanf(line, "FILE %511s", path);
    int fd = open(path, O_RDONLY);
    if (fd >= 0) { do_f2fs_pin(fd, path, 0); close(fd); }
}
fclose(rec);
return 0;
}

static int cmd_list(const char *recpath)
{
    FILE *rec = fopen(recpath, "r");
    if (!rec) { perror(recpath); return 1; }
    char line[1024];
    while (fgets(line, sizeof(line), rec)) fputs(line, stdout);
    fclose(rec);
    return 0;
}

int main(int argc, char **argv)
{
    if (argc < 3) {
        fprintf(stderr,
            "usage:\n"
            " %s pin <record-file> <file>...\n"
            " %s unpin <record-file>\n"
            " %s list <record-file>\n",
            argv[0], argv[0], argv[0]);
        return 1;
    }
    if (strcmp(argv[1], "pin") == 0 && argc >= 4)
        return cmd_pin(argv[2], &argv[3], argc - 3);
    if (strcmp(argv[1], "unpin") == 0) return cmd_unpin(argv[2]);
    if (strcmp(argv[1], "list") == 0) return cmd_list(argv[2]);
    fprintf(stderr, "unknown command: %s\n", argv[1]);
    return 1;
}

```

11.2 fbo_enum.sh

```

#!/system/bin/sh
# fbo_enum.sh - enumerate the static distribution-side files of an
# Android package. Output: one absolute path per line.
#
# Pipe into fbo_pin:
#   fbo_enum.sh com.antutu.ABenchMark | \
#     xargs ./fbo_pin pin /data/local/tmp/fbo_antutu.rec

PKG=$1
[ -z "$PKG" ] && { echo "usage: $0 <package>" >&2; exit 1; }

ABI=$(getprop ro.product.cpu.abi)
case "$ABI" in
    arm64-v8a)   ISA=arm64 ;;
    armeabi-v7a) ISA=arm   ;;
    x86_64)      ISA=x86_64 ;;
    x86)         ISA=x86    ;;
    *)          ISA=$ABI   ;;
esac

APKS=$(pm path "$PKG" 2>/dev/null | sed 's/^package:/' )
[ -z "$APKS" ] && { echo "package not installed: $PKG" >&2; exit 1; }

```

```

DIR=$(dirname "$(echo "$APKS" | head -n1)")

echo "$APKS"
ls "$DIR/oat/$ISA"/* 2>/dev/null
ls "$DIR/lib/$ABI"/* 2>/dev/null

```

11.3 Sample run on an x86 Linux box

Build, prep a sparse test file (so FIEMAP returns multiple extents), and run the full pin / list / unpin cycle:

```

$ gcc -O2 -Wall -o fbo_pin fbo_pin.c
$ dd if=/dev/urandom of=/tmp/fbo_sample bs=4K count=64
$ dd if=/dev/urandom of=/tmp/fbo_sample bs=4K count=32 seek=128 conv=notrunc

$ ./fbo_pin pin /tmp/fbo_sample.rec /tmp/fbo_sample
PIN /tmp/fbo_sample
ufs_PIN lba=0x2db280000 len=0x40000 flags=0x0
ufs_PIN lba=0x2db288000 len=0x20000 flags=0x1
done -- 2 extent(s) pinned

pinned 1 / 1 files; record at /tmp/fbo_sample.rec

$ ./fbo_pin list /tmp/fbo_sample.rec
FILE /tmp/fbo_sample ino=2886497 mtime=1780501629 nextents=2
EXT phys=0x2db280000 len=0x40000 flags=0x0
EXT phys=0x2db288000 len=0x20000 flags=0x1

$ ./fbo_pin unpin /tmp/fbo_sample.rec
UNPIN /tmp/fbo_sample
ufs_UNPIN lba=0x2db280000 len=0x40000 flags=0x0
ufs_UNPIN lba=0x2db288000 len=0x20000 flags=0x1

```

Two extents because the second `dd` left a hole between block 64 and block 128 — exactly the kind of layout you’ll see for installed APKs after the filesystem has aged a bit.

The F2FS pin step printed no warning because we’re on ext4 and the `ioctl` returns `ENOTTY`, which the code intentionally swallows. On an F2FS mount the same code path would actually pin and you’d see the warning only if the kernel refused (e.g., the file was open for write, or the non-GC region was full).

11.4 End-to-end workflow on Android

The intended composition of the two tools on a rooted dev target:

```

adb push fbo_pin fbo_enum.sh /data/local/tmp/
adb shell chmod +x /data/local/tmp/fbo_pin /data/local/tmp/fbo_enum.sh

adb shell
# now on device
cd /data/local/tmp

# Enumerate the file set for one package.
./fbo_enum.sh com.antutu.ABenchMark

# Pin the whole set with one command.
./fbo_enum.sh com.antutu.ABenchMark | \
    xargs ./fbo_pin pin /data/local/tmp/fbo_antutu.rec

# Verify what got recorded.
./fbo_pin list /data/local/tmp/fbo_antutu.rec

# Launch Antutu through the normal launcher. Reads benefit from the
# pin. Nothing of ours is in the data path while Antutu runs.

# When done, unpin.
./fbo_pin unpin /data/local/tmp/fbo_antutu.rec

```

The composition is deliberate: `fbo_enum.sh` is Android-specific, `fbo_pin` is filesystem-and-storage-specific. The seam between them is just absolute file paths on stdout, so the same `fbo_pin` binary serves any workflow that produces a list of files to pin.

11.5 What to change for production

The reference implementation is intentionally minimal. To move from this to a production daemon shipped to OEMs, the deltas are:

1. **`ufs_vendor_pin()`**: replace the `printf` with the actual call to the OEM's pin interface. This is the single most important substitution and is where most of the per-customer integration work happens. The function signature does not change.
2. **Record store**: replace the text record file with something structured (SQLite or a small binary format), and key records by package name so unpin can be requested per-package.
3. **Lifecycle integration**: replace the CLI driver with a long-running service that subscribes to `ACTION_PACKAGE_REPLACED`, `ACTION_PACKAGE_REMOVED`, and an OAT-regeneration signal, and re-pins or unpins accordingly.
4. **Pin region budget management**: when pin region is at capacity, evict the least-recently-launched app's pins before adding a new one. A simple LRU keyed on per-app launch timestamps from `usagestats` is a reasonable first pass.
5. **SELinux policy**: the daemon needs a vendor domain with the capabilities to open every relevant inode under `/data/app`, issue the F2FS pin ioctl, read FIEMAP, and access the UFS pin path.
6. **init.rc service definition**: standard Android service plumbing so the daemon starts at boot and is restarted on crash.
7. **Telemetry**: per-app pin success/failure counts, pin region utilization, time-since-last-pin per app. OEMs need this for tuning.

The core of the implementation — the four-step per-file sequence in `cmd_pin()` — does not change. It is the same sequence whether the caller is a CLI tool, a system service, or a kernel module.